First, remember that a first-class value function can be the value of an object property:

```
yourObject =
  someFunction: ->
```

When a function is accessed as a property of an object and invoked, then the object takes on a special behavior; it becomes `this` inside the function invocation:

```
yourObject.someFunction()
```

If you look at it, because of the presence of the property access, `yourObject` is the object on which the function was invoked. Another way to put it is to say that `yourObject` is the receiver of the function call (it's receiving the current message). That's not quite the whole story, though. Binding can be a little nuanced for the beginner, so to get to a full understanding, it's once again necessary to take a scenic route.

In this section you'll see why a dynamic `this` is useful and how to use it. You'll also see where a dynamic `this` isn't useful and how you can stop `this` from being dynamic by using the fat arrow. That's a head-spinning sentence! Sit back, relax, and take the scenic route. What is `this`?

4.5.1. this

Imagine you have an element in an HTML document and you want to change the contents of the element when it's clicked. This venerable problem, which you've seen before, has been around since the beginning of JavaScript. Here's the minimal HTML5 document:

```
<!doctype html>
<title>How many people are coming?</title>
<body>
<div id='#some-element'>Change this text</div>
```

Now suppose you assign a click handler: